Lab 5: CS527

Release date: 28 August 2026
Submission date: 3rd Sept 2026, 9AM

Problem Statement:

Simulator a mini-computer system.

Users can write a program in a simple programming language, as specified in the language specification section. A compiler will compile the program, and will generate a byte code. Byte code will be read by a computer system, and will initialise a memory. The processor will read the instruction one by one to execute the program, and will modify the memory. At the end, memory will be written to the computer system in a file.

Finally, compilation and computer system should be invoked by a single main file, and executed as a single executable.

Assume all the integer registers in the architecture are of 32 bits. Memory is byte addressable.

In this lab, we introduce vector operations and vector register files. The processor model now has 256 integer registers (x0 to x255) each 32 bit wide and 32 vector registers (v0 to v31) each of 256 bits wide.

Vector operators are: add, sub, multiply, load, store.

In this lab we introduce multi-processing. That is, our computer is able to run multiple programs at the same time. To implement this we introduce a thin OS layer that receives multiple programs via command line interface, and delegates these tasks to the processor.

To avoid saving and retrieving state in the same processor, we use multiple instances of processors and memory. The suggested implementation is:

- Make the register array two dimensional
- Instruction and data memory are also two dimension.
- The number of processors are fixed to NP, NP is constant at compile time.
- All other registers, like PC is also made an array of NP
- Processor implementation receive processor id from the OS, and execute instructions on the registers and memory of the corresponding processor.
- To implement multi-processing, time division multiplexing is implemented. A time slice is kept. OS ask processor to execute a sliced number of instructions. A sleep statement is added to see real time execution

- To show execution of a processor, a new instruction Print is added. Print uses fprintf to print register values to a log file.

OS layer does three tasks: loads a task in the processor, schedules the tasks and take input from end-user for new task

Scheduler:

- Maintains a ready and waiting queue of the tasks - give them IDs
- Maintains task-id (called PID) to processor mapping.
- Run each task in the ready queue for fixed number of cycles
- After executing all the tasks, it execute Shell task, that takes input from the end-user for a new task
- If a task is finished. OS removes that task from the queue
- Similarly, if all the four processors are busy executing, a new task is created, that will kept in waiting queue.

Shell:

- Shell implements a non-blocking version of the I/O read from STDIN. Some thing like this
- Read character by character, whatever is there in STDIN buffer and keeps it in char array, and exits
- When enter character is received, character is parsed
- Progam.txt is compiled into program.byte and data.byte
- Sends program.byte and data.byte file names to loader
- If “exit” is written on shell, shell no longer takes any further input. That means, only remaining tasks will execute on OS.

Expected input at Shell be like following

```
$                                   % user see this prompt all the time
$pro                                % enter not processed yet
$prog1.txt data1.byte  <-|          % After enter is pressed this string will be passed to loader
```

Loader:

- Takes program.byte and data.byte as input.
- Find if a processor is free. Free processor is assigned the new task, PID-ProcID is updated
- Processor is initialized with program.byte and data.byte
- Task is kept in the ready queue.

Some hints are given the document below. Rest need to developed by the student.

In this lab, we will separate logical memory and physical memory. Whatever memory addresses were generated by the compiler and accessed by the processor will be called logical memory. And all the memory accesses will finally be accessed from a physical memory. To support, logical memory-physical memory, following changes are introduced in the system:

Both physical memory and logical memory is divided into pages. Pages of local memory are called pages, and pages in physical memory are called frames. Frame and pages are of same size, and is defined at compile time using #define

OS: In OS a Memory management unit is included, which has following features:

- It maintains an array of page tables. The size of the page table is determined by PAGESIZE
- It maintains a list of free frames. It provides a free frame number, when requested
    - Frame 0 is reserved, and will never be allocated
- Initialise function is modified, as it loads program and data in physical memory frame wise.
- Similarly the finalise function is modified, it returns the pages used by a task.

Scaffolding code is given in the OS section.

Processor:

- Before accessing the memory, first logical to physical memory translation is done.

Language specification:

Arithmetic operations

Language supports following basic math operations : + - / *

Dest Variable = <Operand 1 variable> <operation> <Operand 2 variable>
Dest Variable = <Operand 1 variable> <operation> <constant value>

Example:
x1 = x2 + 10
x1 = x2 - 20
The maximum value of constant could be 255.

Every addition/subtraction operation updates the following four global flags: Z N C V
Z is 1, if result of operation is 0
N is 1, if MSB of result of operation is 1
C is 1, {For addition: if the unsigned result is strictly less than either of the unsigned inputs}. For subtraction: if operand 1 is bigger than operand 2.
V is 1, {for addition: if both operands sign is same, and sign of result is different than operand's sign.} {For subtraction: if the operands have different signs, and the result's sign matches the second operand's original sign}

Vector arithmetic:

- Operators allowed: +, - and *

- If right hand side variable name is (v0 to v31) than operation is a vector operation
    - First operand is always vector
    - The second operand can be integer or constant.
    - Operations are performed on 32 bits, assuming the vector has 8 integer elements.
    - Examples are given below:

v3 = v1 + v4; // for(i=0; i < 8; i++) $v3_i = v1_i + v4_i$
v3 = v1 + 15; // for(i=0; i < 8; i++) $v3_i = v1_i + 15$
v3 = v1 * v4; // for(i=0; i < 8; i++) $v3_i = v1_i * v4_i$
v3 = v1 * x4; // for(i=0; i < 8; i++) $v3_i = v1_i * x4$

Memory operations: Read and Write

Read <variable> <address>
Write <variable> <address> //address is a constant

Support of square brackets.
Dest Variable = [Operand 2 variable] //memory read.
[Dest Variable address] = Operand 2 variable //memory write to given address stored in a variable.

Operand could be a variable or a constant.
Example:
x1 = [x2]
[x2] = x1;
x1 = [0]

The maximum value of constant can be 255.

Example of memory operation on vector variables:
v1 = [x2] // for(i=0; i < 8; i++, x2 = x2 +4) $v1_i$ = [x2]
v1 = [32] // for(i=0, tmp=32; i < 8; i++, tmp = tmp +4) $v1_i$ = [tmp]
[x2] = v1 // for(i=0; i < 8; i++, x2 = x2 +4) [x2] = $v1_i$

Read and Write should be considered legacy instruction. Should not be used in lab 2 onwards - though compiler should still be able to compile them.

Control operations: ~~No control operation supported in today's lab.~~

Language support labels. Rules:

a) Labels have to start with . character

b) They should be the first character of any line (no space or tab should prefix them).
c) The label will be an alphanumeric string of characters. No special symbol.
d) Line having label will have no other expression

Example:
.label1
.loop

Control operations:
B{Suffix} <label>

Suffix given in table below:

Example:
BEQ .loop
BGE .label1

Code	Suffix	Flags	Meaning
0000	EQ	Z set	equal
0001	NE	Z clear	not equal
0010	CS	C set	unsigned higher or same
0011	CC	C clear	unsigned lower
0100	MI	N set	negative
0101	PL	N clear	positive or zero
0110	VS	V set	overflow
0111	VC	V clear	no overflow
1000	HI	C set and Z clear	unsigned higher
1001	LS	C clear or Z set	unsigned lower or same
1010	GE	N equals V	greater or equal
1011	LT	N not equal to V	less than
1100	GT	Z clear AND (N equals V)	greater than
1101	LE	Z set OR (N not equal to V)	less than or equal
1110	AL	(ignored)	always

Table is taken from [REF1]
More information about these condition flags [ARM]

Data movement operation:

<Dest Variable> = <constant value>

Variable names: x0 to x255.
Constant value: 0 to 255

Print operation

A new instruction print is introduced.

Print <Variable>

This instruction will have the following print the following in a log file:

Process id: x<variable> : <value in hex>

Support for comments. Any character following % will be ignored by parser.

Example: Sum of an array. Assume size of Array is stored at address 0, elements stored at 0x4 onwards location. Final result will be stored at address = size * 4 + 4.

```
x1 = 0       % index
x2 = 0       % sum
x3 = 0       % array address
x4 =  [x3]   %  read the size of array
.loopback
x15 = x1 - x4   % compare
BEQ .exit
x3 = x3 + 4    % increase the address
x6 =  [x3]      % load the array data
x2 = x2 + x6  % update sum
x1 = x1 + 1    % increment index
print x1
BAL .loopback
.exit
[x3] = x2
```

Example bytecode output of the above program is **(written in hex)**

```
F 1 0 0
F 2 0 0
F 3 0 0
5 4 0 3
2  F 1 4
10 0 0 6
9 3 3 4
```

5 6 0 3
1 2 2 6
9 1 1 1
1E 0 0 **FA**
6 3 0 2

Byte code specification

Compiler generates a byte code for every operation and finally the byte code is stored in file called “program.byte”.

Format of each line of byte code

<opcode> <dest register> <operand 1> <operand 2>

For branch instructions:

<opcode> <0> <0> <relative offset of jump location>

Relative offset could be positive or negative, such that:

next instruction address = current instruction address + offset.

Operation	Opcode if second operand is a variable	Opcode if second operand is a constant
Integer Add	0x01	0x09
Integer Subtract	0x02	0x0A
Integer Multiply	0x03	0x0B
Divide	0x04	0x0C
Integer Memory read	0x05	0x0C
Integer Memory write	0x06	0x0E
Data movement	0x07	0x0F
Print	0x08	
Branch	0x10 + code given in branch table	
Vector Add	0x21	0x29
Vector Substract	0x22	0x2A
Vector Multiply	0x23	0x2B
Vector memory read	0x25	0x2C

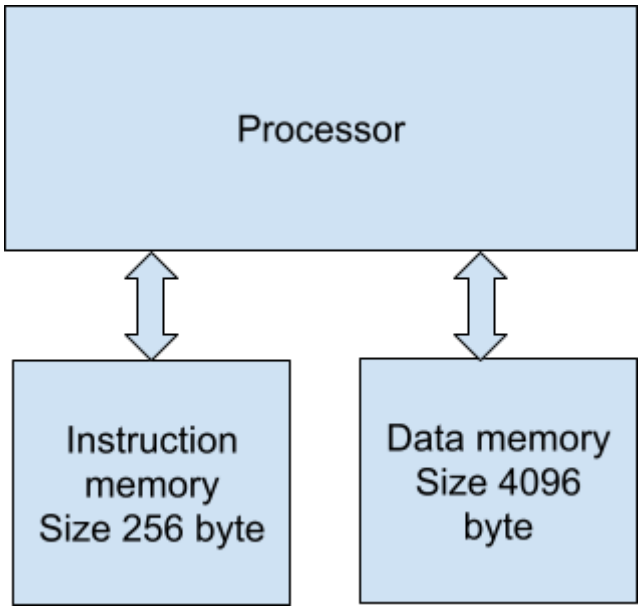
Vector memory write	0x26	0x2E

In case of memory read, write and data movement operations - operand 1 is 0.
In case of Print, Dest register as well as operand 1 is ‘0’, register mentioned in Print instruction will be at operand 2.

Note that

- Every line has four bytes
- The maximum value for each byte is 255.

Processor specification



Processor has four functions:
reset()
fetch()
decode()
execute()

Following data elements are there in processor:
Int Register[NP][256]
Int PC[NP], opcode, dest, src1, src2
extern char Instruction[NP][256], Data[NP][4096];
Int end_of_simulation[NP] = 0

Int proc_id
FILE *fd_log

It has a global array of registers of size 256. And a global variable named PC. Also global flags N Z C V

Reset() initializes each element of register array to 0 as well initialize PC to 0. Open a log file, in append mode.
fetch(): reads four bytes from the PC address of instruction memory and stores in opcode, dest, src1, and src2 respectively. Calls getPhysicallAddress(proc_id, 1, PC) to get physical address. Then access the memory.
decode() : void and empty function at this time
execute(): based on opecode, it performs the operations. Once see 0 opcode, set end_of_simulation = 1. In case of memory read/write operation, it calls getPhysicallAddress(proc_id, 1, address) to get a physical address. Then access the memory.

```
process_instructions(proc_id, instruction_count){
        for( i= 0 to instruction_count) {
                fetch();
                decode();
                execute();
        }
        sleep(10 usec)
}
```

Memory

Memory is coded as a separate ‘c’ file.

```
#define MEMSIZE 8192
#define PAGESIZE 512
char memory[MEMSIZE];

Char Instruction[NP][256];
Char Data[NP][4096];
```

Assumes program.byte and data.byte present in the directory. The memory is byte addressable which means when you read instructions from char array will read the instruction bye-by-byte.

data.byte and program.byte are stored as an array of hex bytes. In each line four bytes in hex format (without prefix 0x) separated by space are stored

For example -1 is represented as
FF FF FF FF

~~initialize() {~~
~~//read the data from "program.byte" and populate Instruction memory~~
~~//read the data from "data.byte" and populate data memory.~~
~~}~~

~~finalize() {~~
~~//Write data.byte~~
~~}~~
// Moved to OS

Operating System (OS)

OS is coded a separate 'c' file.

Data elements in OS:
List of current active processes.
Mapping of processes to processor ids.

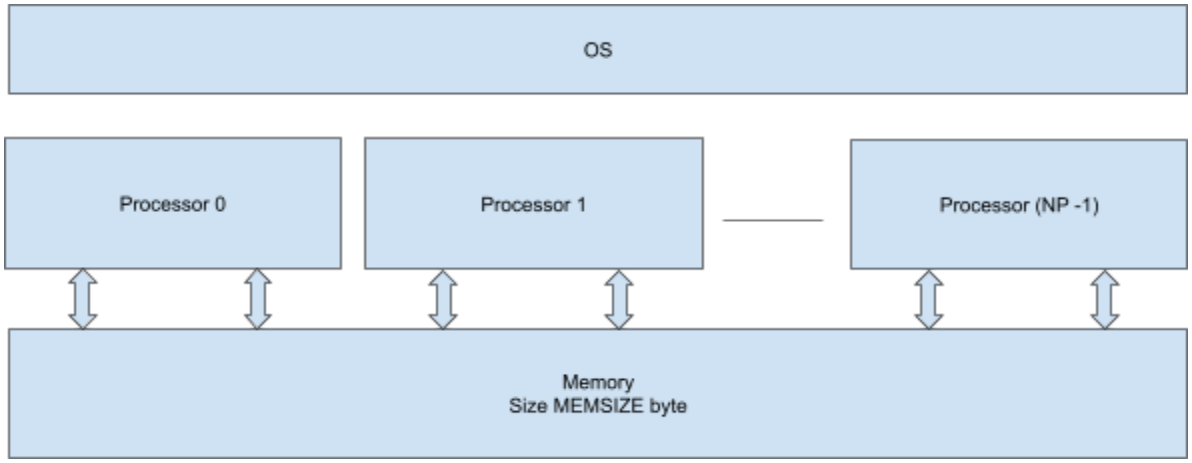
Functions:

```
scheduler ()  {
        //all the processes, along with a shell process will keep getting a turn in round robin style
        for(all the active processes, pid){
                Proc_id = map_pid_proc_id(pid)
                process_instructions(pid, 10);
        }
        shell();
}

//purpose of shell is to receive new processes
shell() {

}

loader() {
}
```

Memory management related functions:

```
//getPhysicallAddress Takes three arguments: proc_id, isFetch = 1 if access is done by fetch
and 0 otherwise, address is logical address. This function returns a physical address.
int getPhysicallAddress(int proc_id, int isFetch, int address) {
        //get the page number
        //find index in page table:
        Index = (isFetch) ? address / PAGESIZE : address /PAGESIZE + 1024/PAGESIZE.
        Access the page table to get physical page number
        Physical address = physical page number * PAGESIZE + address % PAGESIZE

}


char pageTable[MAX_PROC][NUM_LOGICAL_PAGES]
char freePages[NUM_PHYSICAL_PAGES]. Stores 0 initially in all pages, and 1 when page is
allocated.

Page tables size:
    - Defined by page size. If page size is 512, then following calculations work:
        - Number of pages: 1024 / 512 + 4096/ 512 = 10
    - Number of physical pages. Calculated by MEMSIZE. Assuming memsize is 8192, it is 16
Thus the page table can be defined by an array of 10 characters, for page size 512 bytes.

int getFreePage() {
        //OS maintains a list of free pages in Main memory.
        //OS iterate through this list Returns first available free page
        //mark it allocated

        If no free page found, print ERROR and exit from simulation
}


initialize() {
//read the data from “program.byte” and populate Instruction memory

//finds how many PAGES are there in program.byte.
        //for every page, get a physical page using getFreePage()
        //update the physical page number in page table


//read the data from “data.byte” and populate data memory.
//finds how many PAGES are there in data.byte.
        //for every page, get a physical page using getFreePage()
        //update the physical page number in page table
```

```
}

finalize() {
        //Write data.byte
        //reset page table.
        //Free pages from freePage array.
}
```



Main file

```
main(int argc, char** argv) {

compile();   // This will also move to OS, as now program is input to OS
initialise();  //This will also move to OS, as need to initialize a processor everytime a process
                start
reset();       // This will also move to OS

while(end_of_simulation) {  // Now, there has to something similar in OS

}

finalize();     // This will also move to OS,

}
```

It is suggested that the program file is passed as argument to your final executable - using argc and argv.

Directory structure

Following code files should be there:

main.c
compiler.c
compiler.h
processor.c
processor.h
memory.c
memory.h
os.h
os.c

Makefile

Some tutorials on multi-file compilation:
https://www.codewithc.in/tutorials/multi-file-programs
https://www.w3schools.com/c/c_organize_code.php
You can search more on the internet

Makefile tutorial:
https://makefiletutorial.com/

FAQs

Example user level programs:

1. Sum of two arrays A and B of N numbers, the result is stored in another array of N numbers. Though N is given at run time, assume it is always multiple of 8.
    a. Four bytes starting from 0x0 stores N
    b. Next 4N bytes store the Array A, Next 4N bytes store the array B. Result is stored in the next 4N bytes.
    c. Use vector instructions to add these arrays.
2. FIR filter response:

a. Input starts at 0x0 of data memory. The first four bytes tell the size of the array, say N. N is always multiple of 8, but less than or equal to 64. Next 32 bytes store 8 weight values. Next 4N bytes are the input array.
b. Output is stored 0x100 onwards..
c. Use Vector instructions to write code for FIR

Pseudo code of FIR filter

```
for(k=0; k < N-8; k++) {
   sum=0;
   for(i=0; i < 8; i++) {
      sum = sum + input[k+i] * weight[i]
   }
   output[k] = sum;
}
```

Data size

- In this assignment, data size is not specified. ~~You can assume each operation is 8 bits, or 32 bits. 32 bit operation size is preferable.~~ Data size is always 32 bit.

Submission Instructions:

- Submit as a single zipped file - containing all your code, and test programs.